Mojo: Engineering a Lightweight Rust Chess Engine for the Web

sbharga
https://github.com/sbharga/mojo

August 4, 2026

Abstract

Modern chess engines obtain much of their strength from an accumulation of carefully interacting search and evaluation techniques, yet deploying that accumulation in a browser introduces unusual constraints on code size, latency, cancellation, and portability. This paper presents Mojo, a chess engine written in safe Rust and compiled to WebAssembly. The engine combines iterative-deepening principal-variation search with a clustered transposition table, staged move ordering, quiescence search, and several guarded selective search mechanisms. Its handcrafted evaluator uses tapered midgame/endgame scores, incrementally maintained material and piece-square terms, cached pawn structure, nonlinear king danger, and a generated king-and-pawn bitbase. Offline tools fit 887 evaluation parameters from labeled positions and tune a small search-parameter surface through self-play. At the systems boundary, separate browser workers isolate move and analysis searches, atomic stop flags provide prompt cancellation, and baseline and SIMD Wasm artifacts remain below a fixed 250,000-byte gzip limit. On an Intel Core Ultra 5 235 under Node.js, fixed-depth measurements span 2.17–3.79 million nodes per second across four position classes. SIMD changes geometric-mean single-PV throughput by only 0.26% on this platform, illustrating why feature availability and measured speedup should not be conflated. The contribution is an integrated case study of how established chess-search methods can be arranged into a small, reproducible, browser-local engine rather than a claim of a new search algorithm or absolute playing strength.

Keywords: computer chess, alpha–beta search, handcrafted evaluation, Rust, WebAssembly, browser systems

1 Introduction

Chess has served as a compact laboratory for heuristic search since Shannon (1950) separated the problem into a game-tree search and a static evaluation of positions that cannot be searched to completion. That division remains recognizable in contemporary engines, but the surrounding engineering has changed substantially. A practical engine must reuse work, order moves well enough to make alpha–beta cutoffs effective, stabilize tactical leaf positions, allocate time under uncertainty, and validate aggressive selectivity statistically. A browser engine must do so while also coexisting with the event loop, crossing a language boundary, and fitting inside a download budget.

Mojo is a Rust engine compiled to WebAssembly and embedded in a React application (sbharga, 2026). Its design goal is deliberately three-sided: remain lightweight, search quickly under short move budgets, and avoid strength regressions. The first side is made concrete by a hard gzip limit of 250,000 bytes for both baseline and SIMD artifacts. The second is addressed by persistent search state, selective search, incremental evaluation, and adaptive clock polling. The third is addressed by unit tests, deterministic fixed-depth tests, color-swapped self-play, and a sequential test before search-parameter changes are accepted.

This paper makes four contributions. First, it documents the interaction of the engine's search mechanisms at the level of actual guards and data structures, rather than presenting an inventory of chess-programming terms. Second, it describes a compact classical evaluator whose expensive terms are separated into incrementally updated, pawn-cached, and dynamically scanned parts. Third, it explains the concurrency and cancellation choices required to preserve a per-move time contract in the browser. Finally, it reports a reproducible local benchmark that distinguishes node count, throughput, artifact size, and achieved depth. These are systems measurements, not an Elo estimate.

2 Background and Related Work

2.1 Game-tree search

For a side-to-move-relative evaluator, minimax can be written in the symmetric negamax form

$$V(s,d) = \max_{m \in M(s)} -V(T(s,m), d-1), \tag{1}$$

where $M(s)$ is the set of legal moves and T applies a move. Alpha–beta search maintains lower and upper bounds α and β and avoids exploring subtrees that cannot change the root decision. Its effectiveness is dominated by move order; with ideal ordering the effective exponent can be roughly halved (Knuth and Moore, 1975). Principal-variation search (PVS), closely related to NegaScout,

searches the expected best move with a full window and later moves with a null window, re-searching only a move that proves the expectation wrong (Marsland et al., 1987).

Depth-limited evaluation is vulnerable to the horizon effect: a volatile position can conceal a forced exchange just beyond the nominal depth. Quiescence search extends selected tactical continuations until a stand-pat evaluation is more credible (Beal, 1990). Selective search goes further by reducing or pruning branches that are unlikely to influence the bound. Null-move pruning uses the observation that forfeiting a turn is usually worse than making the best legal move, but requires care in zugzwang positions (Donninger, 1993). Futility pruning rejects shallow moves whose optimistic static margin cannot raise alpha (Heinz, 1998); ProbCut uses a reduced search above a margin to predict a full-depth cutoff (Buro, 1995b). Singular extensions take the complementary approach: search deeper when one move appears uniquely necessary (Anantharaman et al., 1990).

Repeated positions make chess a graph rather than a tree. Position hashing originated as a practical board-game technique with Zobrist's tabulation method (Zobrist, 1970). A transposition table stores a searched position's value, depth, bound type, and best move; collision and replacement policy matter because the table is necessarily finite (Breuker et al., 1994). Mojo obtains legal move generation and board hashing from `cozy-chess` 0.3.4 (analog-hors, 2026), while keeping its own rule-aware keys and search tables.

2.2 Evaluation, tuning, and testing

Handcrafted evaluation is commonly a weighted combination of chess features. Statistical fitting of feature weights from labeled positions predates current neural evaluators; logistic regression is attractive because it maps a linear score to an interpretable game-outcome probability (Buro, 1995a). Search parameters are harder to fit because their effect is observed through noisy game outcomes rather than a differentiable loss. Simultaneous perturbation stochastic approximation (SPSA) estimates a multivariate update from only two noisy objective evaluations per iteration (Spall, 1992). Candidate validation can then use a sequential probability ratio test (SPRT), which accumulates evidence until one of two hypotheses is accepted at specified error rates (Wald, 1945).

Exact endgame knowledge provides a useful exception to heuristic evaluation. Retrograde analysis propagates terminal outcomes backward through the state space (Thompson, 1986). Mojo applies the same fixed-point idea to the small king-and-pawn-versus-king (KPK) domain at initialization, rather than shipping a large general tablebase.

3 System Architecture

Figure 1 separates the interactive and offline paths. The Wasm-facing `Engine` accepts FEN plus prior positions, exposes one completed depth at a time, and returns plain principal-variation data. A persistent `SearchCore` keeps its transposition table and learned ordering histories across iterative depths and adjacent positions. The only board mutation occurs in immutable child positions supplied by safe Rust APIs; crate-wide `unsafe_code` is forbidden.

The browser owns two workers. The *move* instance performs the time-critical search that chooses a move. The *analysis* instance may compute MultiPV output continuously. A Wasm call is synchronous within its worker; therefore, placing both workloads behind one instance would allow a queued analysis call to consume a move's nominal budget before the move search started. Isolation makes the budget a property of the move computation rather than of queueing order.

At startup, each worker validates a small SIMD module and chooses either the baseline or `+simd128` artifact. WebAssembly supplies compact, validated, language-independent code and predictable interoperability with a host environment (Haas et al., 2017). Cross-origin-isolated deployments add a `SharedArrayBuffer` stop word. The Rust search polls it with atomic loads between adaptively sized node batches; environments without shared memory fall back to cancellation between completed depths. The design remains functional in both cases, but prompt mid-iteration cancellation requires the appropriate COOP/COEP headers.

4 Search Design

4.1 Iterative deepening and root search

The public analysis operation searches exactly one requested depth, while the worker drives iterative deepening. This division gives JavaScript a safe cancellation and reporting boundary without duplicating search logic. Within one depth, the previous iteration's score centers a 20-centipawn aspiration window. A fail-low or fail-high widens only the failed side; after four retries, the search falls back to the full interval. Mate-range scores bypass aspiration because their distance-sensitive representation should not be clipped around an ordinary centipawn estimate.

At the root, the first move receives a full-window search. Later moves first receive a zero-window PVS probe and are re-searched only when the probe lies between alpha and beta. Root order persists the previous score and subtree node count for each move, placing a previously strong and expensive move early. MultiPV repeats the root search while excluding already selected root moves. Excluded searches do not overwrite the primary root

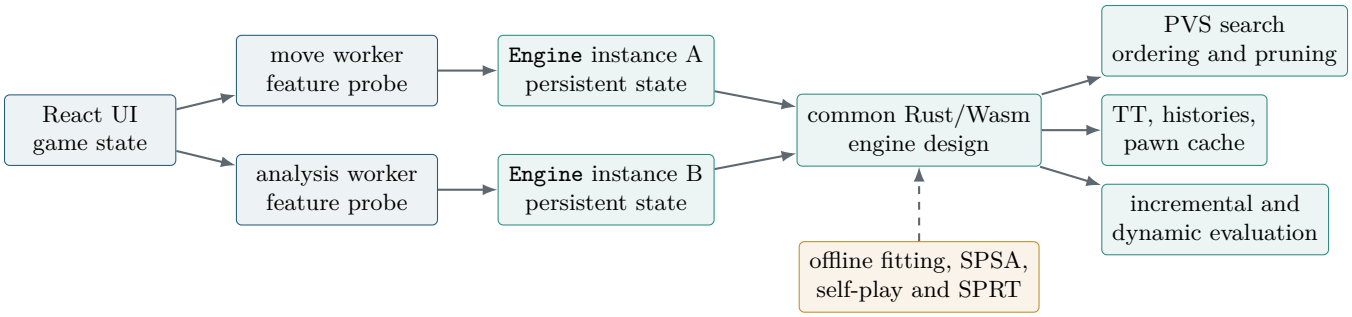


Figure 1: Mojo execution architecture. The move and analysis workers own separate engine instances; the common-design box denotes shared implementation, not shared runtime state. The dashed arrow is an offline configuration path.

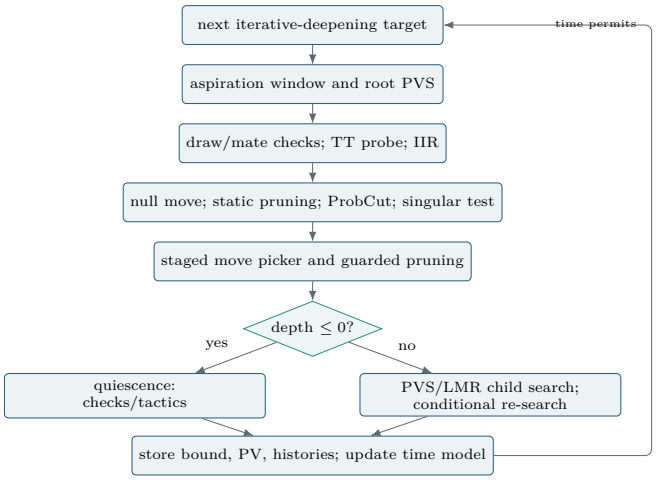


Figure 2: Search control flow. The ordering is significant: cheap bound and static tests precede move construction, while reduced searches are verified when they can change alpha.

transposition entry, preventing a second-best line from displacing the principal move in the next iteration.

The worker stops deepening using more than elapsed wall time. It tracks score stability, how much work concentrated in the best root move, and a smoothed effective branching factor (EBF). The last iteration time and EBF predict the next iteration's cost. A stable, inexpensive decision can stop near half the nominal budget; a score drop or unstable best move allows up to roughly 80%. The EBF gate blocks an iteration predicted not to fit, except when the soft fraction indicates that extra effort is warranted. These policies explain why a larger limit need not increase completed depth in every row of Figure 4: the engine may begin and time out in a deeper iteration while still returning the last complete result.

4.2 Transpositions, draws, and terminal scores

The transposition table contains 2^{18} cache-line-aligned buckets, each holding four 16-byte entries, for 2^{20} entries and 16 MiB total. An entry stores a 64-bit key, encoded move, score, optional static evaluation, signed depth, bound type, and six-bit generation. Exact, lower, and upper bounds are consumed only when their stored depth is sufficient. Mate scores are adjusted by ply on store and load so that transposing into a line preserves preference for a shorter mate or longer defense. Same-key shallow writes do not replace deeper results; collision replacement balances depth and age.

Search keys incorporate both position repetition state and the halfmove clock. The latter is necessary because a visually identical board near the fifty-move boundary has different legal value. The search detects the fifty-move rule, insufficient material, and threefold repetition from real prior positions and the current path. Null moves are marked by a path boundary and cannot create a fictional repetition against real game history. Checkmate scores are $\pm(30000 - p)$ at ply p; stalemate and detected rule draws score zero.

4.3 Ordering and selective search

The staged move picker delays work until it can matter. It returns a legal TT move first, generates tactical moves ordered by victim value and capture history, and runs static exchange evaluation (SEE) only as each tactical move is selected. Non-losing tactics precede killers, the countermove, and a threat move learned from a failed null search. Quiet moves then use main history plus one- and two-ply continuation history; losing captures are deferred to the end. A root-specific picker uses earlier iteration scores and subtree effort.

Late-move reductions are precomputed in a 32-by-64 table with approximately logarithmic growth in depth and move index. Context then modifies the base reduction: PV nodes, checking moves, improving evaluations, and strong history reduce it; cut nodes and poor history

Table 1: Selective mechanisms in Mojo. The safeguards are part of the algorithm: they constrain where a heuristic assumption may discard or reduce work.

Mechanism	Action	Principal safeguards
Null move	Reduced null-window search with depth-dependent reduction; record a threat after fail-low	Non-PV, not in check, non-pawn material, depth ≥ 3, halfmove clock < 90; verification at depth ≥ 10
Reverse futility / razoring	Static fail-high cutoff or shallow fail-low quiescence probe	Non-PV, not in check, non-mate window; bounded to shallow depths and checked for legal moves
ProbCut	Search promising captures at depth $d - 4$ against $\beta + 180$	Depth ≥ 5, non-PV, not in check, SEE-good captures only; store a reduced-depth lower bound
Late-move reduction	Reduce late quiet zero-window probes according to depth and move index	No reduction for captures or evasions; adjust for PV/cut node, checks, history, improving eval; re-search on alpha improvement
Move pruning	SEE, late-move, history, and futility rejection	Non-PV, not in check, not first move, not checking, not a detected threat defense; shallow-depth bounds
Singular extension	Exclusion search extends a uniquely strong TT move by one or two plies	Depth ≥ 7, sufficiently deep lower/exact TT entry, non-mate score, global two-extension budget
Qsearch delta	Skip an exchange that cannot reach alpha even with captured material plus margin	Never while in check; retain promotions and checks; reject SEE-losing captures first

increase it. Any reduced probe that raises alpha is repeated at full depth, followed by a full-window re-search if it remains inside the window. Thus LMR is a speculative allocation of depth, not an unconditional acceptance of a reduced score.

Null-move pruning uses reduction $R = 2 + \lfloor d/4 \rfloor$. At depth ten and above, a null fail-high triggers a verification search from the original position with null pruning disabled. The engine also recovers the null line's threat move after fail-low and promotes defenses against that threat in ordering and pruning. The material and halfmove guards specifically address the zugzwang and rule-boundary failure modes described in the null-move literature (Donninger, 1993).

ProbCut and singular extension make opposite predictions from deeper TT or reduced-search evidence. ProbCut asks whether a good capture already exceeds a raised beta at reduced depth. Singular extension performs an exclusion search without the TT move; a large fail-low marks that move as unique, while multiple viable alternatives can produce a multi-cut or a negative extension. Check extensions and singular extensions share a two-ply per-line budget, preventing unbounded growth.

At depth zero, fail-soft quiescence searches all legal evasions while in check, or captures and promotions otherwise. It retains a stand-pat lower bound only when standing pat is legal. SEE and delta pruning remove losing or insufficient captures, except promotions and moves giving check. Quiescence results are stored as depth-zero TT bounds and can be reused by later probes.

5 Evaluation and Learning

5.1 Tapered handcrafted evaluation

The evaluator returns a side-to-move-relative score compatible with Eq. (1). Material and piece-square values are packed as signed midgame and endgame 16-bit halves of one 32-bit integer. Additions therefore update both components together. With phase $\phi \in [0, 24]$, the white-frame score is

$$E_{\text{white}} = \frac{\phi E_{\text{MG}} + (24 - \phi) E_{\text{EG}}}{24}. \tag{2}$$

Minor pieces contribute one phase unit, rooks two, queens four, and pawns and kings zero. The result is converted to the side-to-move frame, receives a tempo bonus, and is finally scaled for draw proximity.

An `EvalAccumulator` travels with every searched position. Moving a piece subtracts its material and piece-square contribution at the origin and adds the destination contribution; captures, en passant, promotion, and castling have explicit updates. The same accumulator maintains phase and a pawn-only hash. Debug builds compare incremental children against full recomputation across move types.

Terms that depend only on pawn placement are stored in a direct-mapped 128-KiB pawn cache. The cached record contains packed structure score and passed-pawn bitboards. Doubled, isolated, connected, backward, passed, and protected passed pawns are computed here. King-distance bonuses for passers are deliberately excluded because two boards with the same pawn key can have different king locations.

Dynamic scanning walks knights, bishops, rooks, queens, and kings once. It accumulates safe mobility, attack maps, attacks into each king zone, and threat classes. A nonlinear 32-entry king-danger curve combines

the number and type of attackers, attacked king-zone squares, safe checks, pawn shelter, and pawn storms. Additional terms cover the bishop pair, open and semi-open rook files, rooks behind passers or on the seventh rank, minor-piece outposts, hanging pieces, and attacks by pawns or minors on more valuable pieces. Opposite-colored-bishop endings scale the complete score toward zero.

Bare-king endings need a gradient before mate appears in the horizon. The endgame component rewards driving the defending king toward an edge, bringing the attacking king closer, and confining it with a rook. Bishop-and-knight mate instead rewards the corner controlled by the bishop. Exact KPK positions override this guidance. At initialization, a monotone fixed-point procedure classifies $2 \cdot 24 \cdot 64 \cdot 64$ normalized states and packs the winning set into 24 KiB. File and color symmetry map every legal KPK board into that domain; wins receive a large signed score and known draws return zero. This is a small compute-at-initialization instance of retrograde analysis (Thompson, 1986).

The fifty-move rule is integrated twice. Search returns an exact draw at 100 half-moves, while evaluation before that boundary is damped toward zero. This prevents a large heuristic score at half-move 99 from obscuring the immediate rule constraint. The same rule-aware key prevents unsafe TT reuse across different halfmove clocks.

5.2 Correction history

The raw evaluator has repeatable residual error. Three side-indexed correction tables learn bounded adjustments keyed by pawn structure, material counts, and non-pawn piece placement. Together they occupy 192 KiB. At a quiet searched node, an exact or compatible bound trains the residual between corrected static evaluation and search result; capture fail-highs are excluded because their tactical score is a poor training target. A depth-dependent weighted update saturates each entry, and the three table values contribute a bounded centipawn correction. This is online calibration of the existing evaluator, not a replacement value function.

5.3 Offline parameter fitting

The evaluator exposes 887 tunable parameters: piece-square cells and paired midgame/endgame weights for material, pawn structure, mobility, king attack, passers, threats, outposts, and rook features. A training row contains a game result $y \in \{0, \frac{1}{2}, 1\}$ and FEN. The tuner removes non-quiet positions, deduplicates board hashes, averages conflicting labels, and deterministically partitions examples into training and validation sets. Labels are transformed into the side-to-move frame used by the evaluator.

For feature vector x and weight vector w, the predicted result is

$$p(y = 1 \mid x) = \sigma\left(\frac{x^\top w}{K}\right), \qquad K = 400, \tag{3}$$

and gradient descent minimizes cross-entropy plus a small L_2 penalty toward the checked-in weights. Material anchors prevent the fit from changing the scale arbitrarily, and left–right piece-square symmetry is restored before emission. This implements outcome-labeled logistic feature fitting in the tradition of Buro (1995a); the repository names the workflow "Texel tuning." Generated source contains only deltas from readable hand-authored bases.

Six search constants—aspiration width, reverse-futility margin, two futility margins, ProbCut margin, and quiescence delta margin—form a separate SPSA surface. In iteration k, random signs Δ_k define bounded candidates $\theta_k^{\pm} = \theta_k \pm c_k\Delta_k$. A color-swapped self-play match supplies a noisy score difference, and the update follows the simultaneous perturbation estimate (Spall, 1992). Seeds, bounds, schedules, opening count, and time control are recorded. Tuning output remains exploratory until a 128-opening, 100-ms-per-move SPRT accepts the +10 Elo hypothesis with $\alpha = \beta = 0.05$. The acceptance policy follows Wald's sequential testing framework (Wald, 1945); it does not turn a small tuning match into a strength claim.

6 Experimental Method

All measurements use repository revision `aeb6b5b`. The host is an Intel Core Ultra 5 235 with 14 visible cores and 24 MiB L3 cache, running 64-bit Linux under WSL2. Node.js 24.18.0 executes the Wasm modules; the artifacts were produced with Rust 1.96.1 and wasm-pack 0.15.0. Although the host has multiple cores, each engine search is single-threaded. Results therefore describe this Node/V8 environment and must not be generalized to all browsers or machines.

The bundled benchmark defines four FENs: the initial position, a tactical position with castling rights and contact in the center, a representative middlegame, and a rook-and-pawn endgame. Each sample creates a fresh engine so that an earlier position or MultiPV configuration cannot warm the TT. The stop-flag path matches a cross-origin-isolated browser.

For throughput, both baseline and `+simd128` artifacts search iterative depths one through nine. Each position is warmed twice and measured seven times. The harness alternates artifact order by repetition to reduce drift. Node counts, best moves, and final scores must be deterministic; the reported rate is cumulative nodes divided by the median wall time. The graph uses single-PV rows. MultiPV=3 is retained as a cost check.

For budget behavior, the baseline artifact searches at nominal limits of 100, 500, and 1000 ms. Three independent repetitions are summarized by median wall time

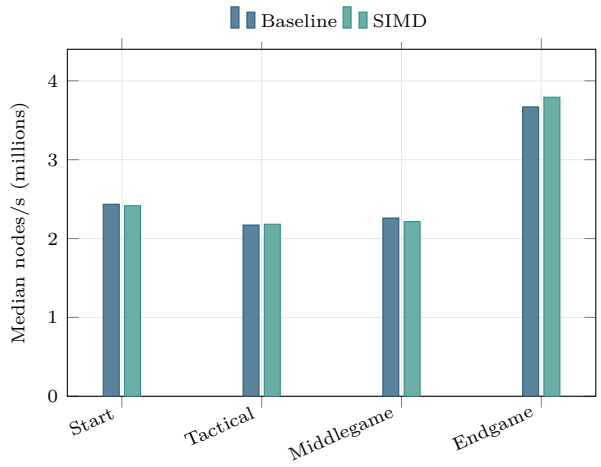


Figure 3: Fixed-depth-9 single-PV throughput. Bars use seven samples after two warmups. Equal node counts make each pair a direct timing comparison.

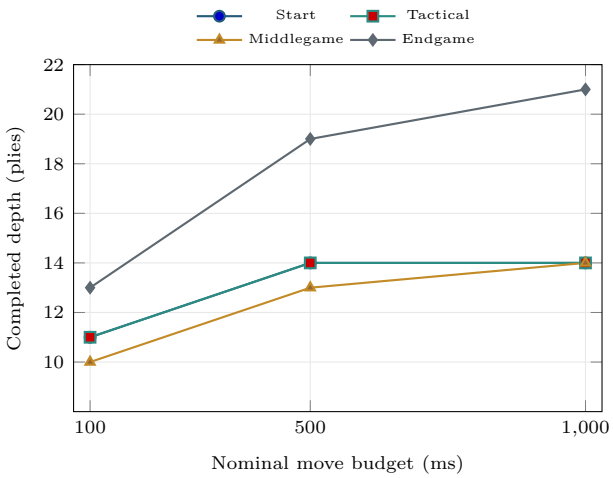


Figure 4: Completed single-PV depth under nominal move budgets. Values are medians of three fresh-engine samples; early stopping and timed-out next iterations are part of the measured policy.

and completed depth. The worker may stop before the limit when its soft-time and EBF tests predict that the next iteration will not fit. It may also spend the limit on an incomplete next depth while reporting the previous complete result. Accordingly, the experiment measures achieved depth, not an assumption that every sample consumes exactly its nominal budget.

Binary measurements use the exact module bytes, gzip, and Brotli quality 11. Initial and peak linear memory are read from the instantiated Wasm memory. The Rust test suite is executed separately in an unoptimized native test profile. The CSV files used by the plots are shipped with the paper, and the PDF is generated directly from them using PGFPlots.

7 Results

Figure 3 shows substantial position dependence. Baseline median throughput ranges from 2.17 million nodes/s in the tactical position to 3.67 million in the endgame. The likely cause is work per node: sliding attacks, move count, pruning eligibility, and cache reuse differ across positions. Nodes/s is therefore a property of a workload, not a machine constant.

SIMD produces small, mixed changes: −0.79% at the start, +0.49% in the tactical position, −1.95% in the middlegame, and +3.35% in the endgame. The geometric mean over the four single-PV rows is +0.26%. Every pair has identical cumulative nodes, best move, and score, isolating code-generation speed rather than search divergence. On this platform the evidence supports shipping a feature-gated variant at low size cost, but not describing SIMD as uniformly faster.

MultiPV illustrates a different cost dimension. At depth nine, baseline cumulative nodes rise from 44,335 to 122,770 at the start, 66,098 to 272,613 in the tactical position, 111,328 to 306,474 in the middlegame, and 29,990 to 92,973 in the endgame. Three lines cost between 2.77 and 4.12 times the single-PV work in this small suite. Isolating background analysis in its own worker is consequently important even when one search is individually fast.

Table 2: Wasm artifact and memory measurements. MiB values use 2^{20} bytes.

Metric	Baseline	SIMD
Raw module (bytes)	1,456,876	1,459,636
gzip (bytes)	223,268	223,553
Brotli-11 (bytes)	132,934	133,266
Initial memory (MiB)	2.56	2.56
Peak memory (MiB)	19.69	19.69

Figure 4 reflects both branching and time management. The endgame reaches depth 13 at 100 ms and depth 21 at 1000 ms, while the other positions reach depths 10–11 and 14, respectively. The start and tactical rows report depth 14 at both 500 and 1000 ms. In those samples the larger budget was spent attempting a deeper iteration that did not complete; the last fully searched line remained depth 14. This behavior is intentional: publishing a partial tree as a completed depth would make depth and score misleading. The engine can return a sound partially searched root line for a timed-out single-PV iteration, but labels it separately and does not substitute it for the completed-depth benchmark field.

Both gzip sizes in Table 2 are below the 250,000-byte ceiling. The SIMD module adds 2,760 raw bytes but only 285 gzip bytes. Peak linear memory is dominated by the 16-MiB TT plus ordering, correction, pawn-cache, board, and Wasm runtime storage. Important fixed structures include a 288-KiB continuation-history table, a 192-KiB

correction history, a 128-KiB pawn cache, a 24-KiB KPK bitbase, a 9-KiB capture history, and a 2-KiB LMR table. These sizes are asserted in tests where layout is part of the design.

The native Rust suite passes 75 tests. Coverage includes standard perft positions, legal PVs, forced mate, repetition and fifty-move draws, incremental evaluation agreement, KPK normalization, SEE thresholds, TT collision and mate-score behavior, every move-picker stage, pruning guards, timeout semantics, clock calibration, MultiPV distinctness, and memory footprints. Tests cannot establish playing strength, but they protect invariants that selective search could otherwise violate silently.

8 Comparison with Stockfish 18

To contextualize Mojo within the modern chess-engine landscape, this section compares its architecture, evaluation model, search selectivity, and runtime constraints with Stockfish 18 (Stockfish Developers, 2026), the premier open-source tournament engine, and its WebAssembly port Stockfish.js (Fiekas et al., 2026). In the accompanying application, Stockfish 18 is bundled as a configurable opponent and reference rather than a direct peer competitor. The two engines occupy fundamentally different points in the design space, as summarized in Table 3.

8.1 Design objectives and resource budget

Stockfish 18 is engineered for maximal playing strength in competitive tournament play and deep analytical search. Its optimization objective is unconstrained by network delivery size, relying on multi-megabyte neural network weights, large dynamic memory allocations, and multi-threaded SMP scaling across physical CPU cores. In contrast, Mojo is engineered to operate under strict browser delivery constraints. Its primary requirements are:

(i) a hard gzip payload ceiling under 250 kB (223 kB actual), enabling sub-second client-side delivery without external asset downloads;
(ii) instantaneous startup with zero network weight transfers;
(iii) deterministic, low-latency search within short interactive budgets (100–1000 ms); and
(iv) strict memory and search isolation across dedicated Web Workers.

8.2 Evaluation paradigm: HCE vs. NNUE

The most significant architectural divergence lies in position evaluation. Stockfish 18 utilizes an Efficiently Updatable Neural Network (NNUE) architecture (Nasu, 2018), employing dual large/small networks or deeply quantized accumulators trained over millions of self-play positions. While NNUE delivers unmatched positional and tactical accuracy, evaluation inference is the dominant computational cost per node, heavily utilizing SIMD vector intrinsics and requiring megabytes of pre-trained parameters.

Mojo deliberately retains a classical handcrafted evaluation (HCE) model. Its tapered midgame/endgame terms, 887 parameters fitted via logistic regression (Texel tuning), 128-KiB pawn cache, dynamic king-danger tables, and generated 24-KiB KPK retrograde bitbase incur zero external network or asset dependencies. Static evaluation in Mojo is purely bitboard arithmetic, leaving the vast majority of CPU cycles available for fast alpha–beta tree traversal. Online correction tables (192 KiB) dynamically compensate for systematic residual evaluation error during search without expanding the static feature footprint.

8.3 Search selectivity and concurrency

Both engines employ iterative-deepening Principal Variation Search (PVS) with aspiration windows, transposition tables, quiescence search, null-move pruning, late move reductions (LMR), and singular extensions. However, their pruning and concurrency strategies reflect their target hardware:

- **Selectivity Depth**: Stockfish 18 applies aggressive, highly tuned pruning cascades (e.g., multi-history heuristics at 1/2/4/6 plies, multi-cut, dynamic double/triple singular extensions, and complex non-linear reduction curves) tuned through massive distributed self-play on Fishtest. Mojo enforces bounded, guarded heuristics (such as a strict two-ply extension budget, material/halfmove null-move guards, and conservative LMR tables; Table 1) to avoid search instability and horizon blunders under short single-threaded time controls.
- **Threading and Concurrency**: Stockfish natively relies on Lazy SMP to scale search across dozens of threads. Mojo implements a single-threaded search kernel per worker, allocating separate Web Workers for move generation and background MultiPV analysis. This guarantees that user interactions remain responsive and move search latency remains strictly bounded regardless of background analysis queueing.

8.4 Web runtime and systems integration

In the browser runtime, Stockfish 18 (Stockfish.js) is compiled from C++20 via Emscripten and controlled asynchronously through the standard Universal Chess Interface (UCI) text protocol over worker `postMessage` channels. Configurable play is achieved by throttling strength via `UCI_LimitStrength` and `UCI_Elo` (clamped between 1320 and 3190 in the application) by introducing

Table 3: Systematic comparison between Mojo and Stockfish 18 (including Stockfish.js).

Dimension	Mojo	Stockfish 18 (Stockfish.js)
Primary Goal	Lightweight, instant-load browser engine with bounded per-move latency	Maximal tournament playing strength and deep analytical depth
Implementation	Safe Rust (`#![forbid(unsafe_code)]`), `wasm-pack`	C++20, Emscripten
Evaluation	Handcrafted tapered evaluation (HCE), 887 Texel-tuned weights, pawn cache, KPK bitbase, correction history	Efficiently Updatable Neural Network (NNUE), dual/large network architectures
Delivery Size	$<$ 250 kB gzip (223 kB gzip, 133 kB Brotli, 1.46 MB raw)	Multi-megabyte lite WASM bundle; 30–70+ MB for full embedded NNUE models
Runtime Memory	$\approx$ 20 MiB peak (2^{20}-entry 16-MiB TT, caches, history)	Tens to hundreds of MiB (large TT, neural net buffers, SMP overhead)
Concurrency	Single-threaded per worker; isolated move/analysis workers	Multi-threaded Lazy SMP across available CPU cores
Cancellation	Adaptively polled `SharedArrayBuffer` atomic flag	Asynchronous UCI `stop` command over Web Worker messaging
Host Interface	Direct typed Wasm-bindgen interface	Standard UCI text protocol (`stdin`/`stdout`)
Strength Profile	Single optimized fast-search profile	Configurable rating emulation via `UCI_Elo` (1320–3190) and unconstrained master strength ($>$ 3600)

Table 4: Single-threaded Wasm throughput and search depth comparison between Mojo and Stockfish 18 Lite under Node.js.

Position	Depth-9 Nodes/s		500-ms Depth	
	Mojo	SF18	Mojo	SF18
Start	893,784	359,384	13	19
Tactical	1,123,105	921,272	12	18
Middlegame	1,416,233	708,000	12	20
Endgame	2,084,637	930,333	18	14

Table 5: Head-to-head match results of Mojo against calibrated Stockfish 18 Elo tiers across balanced opening pairs.

SF Elo	Games	W–D–L	Score	ΔElo	Mojo Perf.
1600	20	19–1–0	97.5%	+636	2236
1800	20	20–0–0	100.0%	+798	2598
2000	20	18–1–1	92.5%	+436	2436
2200	20	15–5–0	87.5%	+338	2538
2400	20	12–5–3	72.5%	+168	2568

intentional search noise and time reductions.

In contrast, Mojo is written in safe Rust (`#![forbid(unsafe_code)]`) and compiled directly to Wasm via `wasm-pack`. Rather than parsing textual UCI streams on the hot path, its JS-to-Wasm bridge passes structured typed data. For cancellation, Mojo employs a `SharedArrayBuffer`-backed atomic stop flag polled adaptively during search, achieving immediate preemption without terminating or reinitializing the worker context.

8.5 Empirical performance and relative Elo

To quantify the runtime performance trade-offs, Table 4 reports single-threaded WebAssembly throughput and achieved search depths across the four standard benchmark positions on the Intel Core Ultra 5 235 test host under Node.js. At fixed depth nine, Mojo delivers 1.22$\times$ to 2.49$\times$ higher node throughput (geometric mean 1.91$\times$) than Stockfish 18 Lite WASM. This throughput advantage stems directly from evaluation complexity: Mojo's handcrafted bitboard arithmetic evaluates in nanoseconds, whereas Stockfish's embedded NNUE network evaluates quantized neural layers at non-terminal nodes.

Conversely, under a fixed 500-ms time control, Stockfish 18 attains greater nominal search depth in middlegame positions (depths 18–20 vs. 12–13 for Mojo) because its neural evaluation enables substantially more aggressive speculative pruning and larger late-move reductions without sacrificing tactical soundness. In the endgame, Mojo's low per-node overhead and dedicated KPK bitbase allow it to achieve depth 18 compared to Stockfish's depth 14.

To evaluate playing strength relative to a standardized scale, Mojo was matched against Stockfish 18 across 100 games (20 games per tier using 10 color-swapped opening pairs from the balanced suite) with Stockfish's `UCI_LimitStrength` calibrated from 1600 to 2400 Elo. As shown in Table 5, Mojo decisively outperforms calibrated tiers up to 2000 Elo ($>$ 92% score) and maintains a winning record against Elo 2200 (15–5–0, 87.5%) and Elo 2400 (12–5–3, 72.5%). Using the standard logistic Elo formula $\Delta\text{Elo} = -400\log_{10}(1/S - 1)$, Mojo demonstrates a consistent empirical performance rating of $\approx$ 2500–2570

Elo relative to Stockfish's calibrated rating scale under fast browser-style time controls.

9 Discussion and Limitations

The implementation demonstrates that compactness and sophistication are not opposites. Most search features are small control policies over shared data: the TT move feeds root order, PVS, singular tests, and quiescence; a null search can yield either a cutoff or an ordering threat; history affects ordering, pruning, and LMR. This reuse keeps the binary small, but it also makes interactions difficult to reason about in isolation. Guard-focused tests and fixed-depth determinism are therefore as important as tests of individual score formulas.

The evaluator makes a similar decomposition by update frequency. Material, piece-square score, phase, and pawn key change incrementally with every move. Pawn relations are cached across boards with the same pawns. Mobility, king safety, and tactical threats are recomputed because their occupancy-dependent attack sets change broadly. This division avoids both a full evaluator at every node and a fragile attempt to update every dynamic relation.

The benchmark has several limitations. Four hand-selected positions do not represent the distribution of a game, and nodes have unequal cost. Median timing on one Node/V8 build does not characterize Chrome, Firefox, Safari, mobile CPUs, thermal throttling, or native Rust. The SIMD comparison measures two compiler artifacts but does not attribute changes to particular Wasm instructions. The timed experiment has only three repetitions and reports completed integer depths, a coarse outcome. No confidence intervals are claimed.

Most importantly, throughput is not strength. A pruning change can search fewer nodes and play worse; a richer evaluator can search fewer nodes per second and play better. While Table 5 characterizes relative performance against Stockfish's calibrated rating scale, this paper avoids claiming a universal Elo rating because external tournament pools, diverse hardware architectures, and long-horizon time controls were not evaluated. The repository's color-swapped SPRT process is suitable for accepting a candidate relative to a fixed baseline, not assigning an absolute tournament standing.

Future evaluation should expand across broader multi-engine pools, diverse mobile and desktop browser runtimes, and long-time-control tournament fixtures. Search ablations would quantify each heuristic's node and strength contribution, though such experiments must retune interacting constants or explicitly acknowledge that they measure a perturbed configuration. Hardware performance counters are not available in ordinary browser execution, so native profiling can complement, but not replace, deployed Wasm measurements.

10 Conclusion

Mojo integrates established computer-chess techniques into a constrained browser system. Its search combines PVS, transposition reuse, staged ordering, quiescence, and guarded selectivity; its evaluator combines incremental tapered scores, cached pawn knowledge, dynamic attacks, learned correction, and an exact KPK exception. Separate workers and atomic cancellation preserve interactive time semantics, while dual Wasm artifacts remain under a strict download limit. Local measurements show deterministic fixed-depth behavior, position-dependent throughput, and high search efficiency relative to Stockfish 18 Lite WASM (1.91× throughput, ≈ 2500–2570 relative Elo). The broader lesson is methodological: report search work, runtime, size, and strength as distinct quantities, and validate each with an experiment designed for that quantity.

References

analog-hors. cozy-chess: Chess move generation and position representation for Rust. Software, version 0.3.4, 2026. URL https://github.com/analog-hors/cozy-chess. Accessed 2026-08-04.

Thomas Anantharaman, Murray S. Campbell, and Fenghsiung Hsu. Singular extensions: Adding selectivity to brute-force searching. *Artificial Intelligence*, 43(1): 99–109, 1990. doi:10.1016/0004-3702(90)90073-9.

Don F. Beal. A generalised quiescence search algorithm. *Artificial Intelligence*, 43(1):85–98, 1990. doi:10.1016/0004-3702(90)90072-8.

Dennis M. Breuker, Jos W. H. M. Uiterwijk, and H. Jaap van den Herik. Replacement schemes for transposition tables. *ICCA Journal*, 17(4):183–193, 1994. doi:10.3233/ICG-1994-17402.

Michael Buro. Statistical feature combination for the evaluation of game positions. *Journal of Artificial Intelligence Research*, 3:373–382, 1995a. doi:10.1613/JAIR.179.

Michael Buro. ProbCut: An effective selective extension of the alpha-beta algorithm. *ICCA Journal*, 18(2): 71–76, 1995b. doi:10.3233/ICG-1995-18202.

Christian Donninger. Null move and deep search: Selective-search heuristics for obtuse chess programs. *ICCA Journal*, 16(3):137–143, 1993. doi:10.3233/ICG-1993-16304.

Niklas Fiekas, nmrugg, and Stockfish Developers. Stockfish.js: WebAssembly and JavaScript port of the Stockfish chess engine. Software, version 18.0.8, 2026. URL https://github.com/nmrugg/stockfish.js.

Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and J. F. Bastien. Bringing the web up to speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 185–200. ACM, 2017. doi:10.1145/3062341.3062363.

Ernst A. Heinz. Extended futility pruning. *ICCA Journal*, 21(2):75–83, 1998. doi:10.3233/ICG-1998-21202.

Donald E. Knuth and Ronald W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4): 293–326, 1975. doi:10.1016/0004-3702(75)90019-3.

T. Anthony Marsland, Alexander Reinefeld, and Jonathan Schaeffer. Low overhead alternatives to SSS*. *Artificial Intelligence*, 31(2):185–199, 1987. URL https://webdocs.cs.ualberta.ca/~tony/OldPapers/ai87.pdf.

Yu Nasu. NNUE: Efficiently updatable neural-network-based evaluation functions for computer shogi. Computer Shogi Association, 2018.

sbharga. Mojo: A rust and WebAssembly chess engine. Software, revision aeb6b5b, 2026. URL https://github.com/sbharga/mojo. Accessed 2026-08-04.

Claude E. Shannon. Programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314):256–275, 1950. doi:10.1080/14786445008521796.

James C. Spall. Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE Transactions on Automatic Control*, 37(3): 332–341, 1992. doi:10.1109/9.119632.

Stockfish Developers. Stockfish: A strong open-source chess engine. Software, 2026. URL https://stockfishchess.org.

Ken Thompson. Retrograde analysis of certain endgames. *ICCA Journal*, 9(3):131–139, 1986. doi:10.3233/ICG-1986-9302.

Abraham Wald. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2):117–186, 1945. doi:10.1214/aoms/1177731118.

Albert L. Zobrist. A new hashing method with application for game playing. Technical Report 88, Computer Sciences Department, University of Wisconsin–Madison, 1970. URL https://research.cs.wisc.edu/techreports/viewreport.php?report=88.